LAPORAN PRAKTIKUM

Prak. Algoritma dan Struktur Data

DISUSUN OLEH:
ZULFAN SAHPUTRA
2025573010031
TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum ini dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan pada mata kuliah Praktikum Algoritma dan Struktur Data.

Dalam proses penyusunan laporan praktikum ini, penulis memperoleh banyak pengetahuan serta pengalaman baru, khususnya dalam memahami lingkungan kerja laboratorium serta penggunaan teknologi pendukung pengembangan web. Oleh karena itu, laporan ini diharapkan dapat memberikan gambaran mengenai proses praktikum yang telah dilakukan serta menjadi bahan pembelajaran bagi penulis maupun pembaca.

Penulis juga menyampaikan ucapan terima kasih kepada Bapak Muhammad Reza Zulman, S.ST., M.Sc. selaku dosen pengampu mata kuliah Praktikum Algoritma dan Struktur Data yang telah memberikan arahan, bimbingan, serta ilmu selama kegiatan perkuliahan dan praktikum berlangsung.

Penulis menyadari bahwa laporan praktikum ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang.

Akhir kata, penulis berharap laporan praktikum ini dapat memberikan manfaat serta menambah wawasan bagi para pembaca.

Buket Rata, 22 April 2026

Zulfan Sahputra

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi algoritma merupakan hal yang sangat penting. Seiring bertambahnya jumlah data, algoritma yang tidak efisien dapat menyebabkan waktu eksekusi menjadi sangat lama bahkan tidak dapat digunakan dalam skala besar. Oleh karena itu, diperlukan suatu metode untuk menganalisis performa algoritma, yaitu menggunakan **Big O Notation**.

Big O digunakan untuk mengukur seberapa cepat waktu eksekusi (time complexity) dan penggunaan memori (space complexity) suatu algoritma terhadap pertambahan input. Pemahaman ini sangat penting dalam menentukan apakah suatu algoritma layak digunakan atau perlu diperbaiki.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami konsep kompleksitas algoritma serta dapat membandingkan efisiensi berbagai algoritma.

1.2 Rumusan Masalah

Adapun rumusan masalah dalam praktikum ini adalah:

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Bagaimana cara menganalisis time complexity dan space complexity?
3. Apa saja jenis-jenis Big O Notation?
4. Bagaimana membandingkan efisiensi beberapa algoritma?
5. Bagaimana cara mengoptimalkan algoritma yang tidak efisien?

1.3 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar kompleksitas algoritma.
2. Menjelaskan notasi Big O.
3. Menganalisis time complexity dan space complexity.
4. Membandingkan beberapa algoritma berdasarkan efisiensinya.
5. Melakukan optimasi algoritma agar lebih efisien.

1.4 Manfaat Praktikum

Manfaat dari praktikum ini antara lain:

1. Mahasiswa dapat memahami efisiensi algoritma.
2. Mahasiswa mampu memilih algoritma yang tepat.
3. Meningkatkan kemampuan analisis dalam pemrograman.
4. Membantu dalam pengembangan aplikasi yang optimal.

BAB II
DASAR TEORI

2.1 Pengertian

Kompleksitas algoritma adalah ukuran yang digunakan untuk mengetahui seberapa efisien suatu algoritma dalam menyelesaikan suatu masalah.

Terdapat dua jenis kompleksitas utama:

- Time Complexity: waktu yang dibutuhkan algoritma.
- Space Complexity: memori yang digunakan algoritma.

Big O Notation digunakan untuk menggambarkan pertumbuhan kompleksitas terhadap input (n).

Beberapa jenis Big O:

- **O(1)** → Konstan (sangat cepat)
- **O(log n)** → Logaritmik
- **O(n)** → Linear
- **O(n log n)** → Linearitmik
- **O(n^2)** → Kuadratik
- **O(2^n)** → Eksponensial (sangat lambat)

Semakin kecil kompleksitas, semakin efisien algoritma tersebut.

2.2 Kesimpulan Dasar Teori

Berdasarkan teori yang dipelajari:

- Big O digunakan untuk mengukur efisiensi algoritma.
- Kompleksitas tidak bergantung pada waktu nyata, tetapi pada pertumbuhan input.
- Algoritma yang baik adalah yang memiliki kompleksitas rendah.
- Penggunaan struktur data seperti **Set** atau **Map** dapat meningkatkan efisiensi.

BAB III
PRAKTIKUM

3.1 Langkah Langkah Praktikum

1. Langkah pertama yang dilakukan adalah membuat folder baru dengan nama **praktikum-5** sebagai tempat penyimpanan file yang akan digunakan selama praktikum.

2. Selanjutnya, folder **praktikum-5** dibuka menggunakan aplikasi Visual Studio Code agar dapat digunakan untuk menulis serta mengelola kode program yang akan dibuat.

3. Setelah itu, dibuat sebuah file JavaScript di dalam folder tersebut dengan nama **touch 01-intro-kompleksitas.js** sebagai tempat untuk menuliskan materi tentang variabel.

4. Kemudian, kode program yang berkaitan dengan variabel dituliskan ke dalam file **touch 01-intro-kompleksitas.js** dengan mengikuti panduan yang terdapat pada modul praktikum.

Code:

```javascript
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris
// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; // ukuran data: 100 ribu
// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}
// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}
// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);
console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

Ouput:

```
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 11 ms
O(n2) cariPasangan : 67 ms

Hasil sama? true
```

5. Selanjutnya buat file JavaScript dengan nama **touch 02-destructuring-spread.js** di dalam folder **praktikum-5** yang digunakan untuk menuliskan materi mengenai tipe data sesuai dengan panduan praktikum.

6. Pada tahap ini, penulisan kode program mengenai tipe data dilakukan di dalam file **touch 02-destructuring-spread.js** berdasarkan petunjuk yang ada pada modul praktikum.

Code:

```javascript
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O
// ── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr) {
  return arr[0];
} // O(1)
function tambahkanItem(arr, item) {
  arr.push(item);
} // O(1)
function isGenap(n) {
  return n % 2 === 0;
} // O(1)
// ── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
// ── O(n) — Linear ───────────────────────────────
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
// ── O(n2) — Kuadratik ───────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}
// ── O(2n) — Eksponensial ────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}
// ── Demonstrasi ─────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
console.log("");
// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

Output:

```
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 998

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
189 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Latihan 1 Identifikasi Kompleksitas

Code:

```javascript
// LATIHAN 1: IDENTIFIKASI KOMPLEKSITAS

// Fungsi untuk menghitung waktu eksekusi
function hitungKompleksitas(n, fn, nama) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(`${nama} → Waktu: ${end - start} ms`);
}

// ========================================
// Fungsi A
// Big-O: O(1)
// Alasan: hanya 1 operasi, tidak tergantung n
// ========================================
function A(n) {
  return n * 2;
}

// ========================================
// Fungsi B
// Big-O: O(n²)
// Alasan: 2 loop bersarang (n × n)
// ========================================
function B(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // sengaja dikosongkan agar tidak lambat karena console.log
    }
  }
}

// ========================================
// Fungsi C
// Big-O: O(log n)
// Alasan: i dikali 2 setiap iterasi
// ========================================
```

```javascript
function C(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// ========================================
// Fungsi D
// Big-O: O(n³)
// Alasan: 3 loop bersarang
// ========================================
function D(n) {
  const arr = new Array(n).fill(0);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        // console.log(x, y, z);
      });
    });
  });
}

// ========================================
// MAIN PROGRAM
// ========================================

const n = 1000;

console.log("=== HASIL PENGUJIAN ===");

// Fungsi A
hitungKompleksitas(n, A, "Fungsi A (O(1))");

// Fungsi B
hitungKompleksitas(n, B, "Fungsi B (O(n^2))");

// Fungsi C
hitungKompleksitas(n, C, "Fungsi C (O(log n))");

// Fungsi D sangat berat jika n=1000
// jadi pakai n kecil agar tidak hang
hitungKompleksitas(100, D, "Fungsi D (O(n^3))");
```

Output:

```
=== HASIL PENGUJIAN ===
Fungsi A (O(1)) → Waktu: 0 ms
Fungsi B (O(n^2)) → Waktu: 16 ms
Fungsi C (O(log n)) → Waktu: 1 ms
Fungsi D (O(n^3)) → Waktu: 5 ms
```

7. Pada tahap ini, buat sebuah file JavaScript bernama **touch 03-space-complexity.js**di dalam folder **praktikum-5** yang digunakan untuk mengerjakan materi tentang operator berdasarkan panduan modul.

8. Selanjutnya dilakukan penulisan kode program mengenai operator ke dalam file **touch 03-space-complexity.js** berdasarkan materi yang tersedia. Kemudian, latihan yang ada di bagian akhir file ditambahkan dan dikerjakan sesuai dengan instruksi pada modul praktikum.

Code:

```javascript
// 03-space-complexity.js
// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}
// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}
// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemen
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
```

Output:

```
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
```

Latihan 3 Class BankAccount

Code:

```javascript
// 1. Fungsi Lambat (Nested Loop)
// Big-O Time  : O(n²)
// Big-O Space : O(1)
// Alasan:
// - Dua loop bersarang → n * n
// - Tidak menggunakan struktur data tambahan
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// 2. Fungsi Cepat (Menggunakan Set)
// Big-O Time  : O(n)
// Big-O Space : O(n)
// Alasan:
// - Hanya 1 loop
// - Set untuk lookup cepat (O(1))
function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let i = 0; i < arr.length; i++) {
    const angka = arr[i];
    const pasangan = target - angka;

    if (seen.has(pasangan)) {
      return [pasangan, angka];
    }

    seen.add(angka);
  }

  return null;
}

// ========================================
// Fungsi untuk ukur waktu
function ukurWaktu(fn, arr, target, nama) {
  const start = Date.now();
  const hasil = fn(arr, target);
  const end = Date.now();

  console.log(`${nama} → Hasil:`, hasil, `| Waktu: ${end - start} ms`);
}

// ========================================
// 3. Pengujian kecil
const dataUji = [2, 7, 11, 15];
const targetUji = 9;

console.log("=== UJI KECIL ===");
ukurWaktu(cariPasanganLambat, dataUji, targetUji, "Lambat");
ukurWaktu(cariPasanganCepat, dataUji, targetUji, "Cepat");

// ========================================
// 4. Pengujian besar (50.000 data)
function generateArray(n) {
  const arr = [];
  for (let i = 0; i < n; i++) {
    arr.push(Math.floor(Math.random() * 100000));
  }
  return arr;
}

const dataBesar = generateArray(50000);
const targetBesar = -1; // kemungkinan besar tidak ada
```

```javascript
console.log("\n=== UJI BESAR (50.000 DATA) ===");

// Lambat bisa sangat lama → optional
// ukurWaktu(cariPasanganLambat, dataBesar, targetBesar, "Lambat");

ukurWaktu(cariPasanganCepat, dataBesar, targetBesar, "Cepat");

// ========================================
// 5. DISKUSI
/*
Fungsi mana lebih baik?
→ cariPasanganCepat lebih baik

Trade-off:
- Cepat:
  + Time lebih cepat (O(n))
  - Butuh memori tambahan (O(n))

- Lambat:
  + Tidak butuh memori tambahan (O(1))
  - Sangat lambat untuk data besar (O(n²))

Kesimpulan:
Gunakan versi cepat jika data besar dan memori cukup.
Gunakan versi lambat jika memori terbatas.
*/
```

Output:

```
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
=== UJI KECIL ===
Lambat → Hasil: [ 2, 7 ] | Waktu: 0 ms
Cepat → Hasil: [ 2, 7 ] | Waktu: 0 ms

=== UJI BESAR (50.000 DATA) ===
Cepat → Hasil: null | Waktu: 12 ms
```

9. Pada tahap berikutnya, buat sebuah file JavaScript bernama **touch 04-refactor.js** di dalam folder **praktikum-5** yang digunakan untuk mempelajari materi tentang percabangan berdasarkan panduan modul praktikum.

Code:

```javascript
// 05-refactor.js — Refactoring kode berperforma buruk
// ═══════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════
// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}
// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
// ═══════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════
// BURUK: O(n2) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}
// ── Benchmark ──────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

Output:

```
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

10. Pada tahap ini, buat folder baru bernama **tugas** di dalam direktori **praktikum-5** yang berfungsi untuk menyimpan seluruh file yang berkaitan dengan tugas praktikum.

11. Pada langkah berikutnya, buat sebuah file JavaScript di dalam folder **tugas** dengan nama **tugas** yang digunakan untuk menuliskan program latihan sesuai dengan instruksi praktikum.

12. Pada tahap ini, penulisan kode program dilakukan di dalam file **tugas-1.js** berdasarkan ketentuan tugas yang telah diberikan.

Code:

```javascript
// Helper waktu
const ukur = (fn, ...args) => {
  const t0 = Date.now();
  const res = fn(...args);
  return { res, time: Date.now() - t0 };
};

// ========================================
// A. INTERSECTION
// O(n^2), O(1)
const interN2 = (a, b) => {
  const r = [];
  for (let x of a) for (let y of b) if (x === y && !r.includes(x)) r.push(x);
  return r;
};

// O(n), O(n)
const interSet = (a, b) => {
  const s = new Set(b);
  return [...new Set(a.filter((x) => s.has(x)))];
};

// ========================================
// B. ANAGRAM
// O(n * k log k), O(n)
const groupAnagram = (arr) => {
  const m = {};
  for (let w of arr) {
    const k = [...w].sort().join("");
    (m[k] ||= []).push(w);
  }
  return Object.values(m);
};

// ========================================
// C. 2 angka = kuadrat angka lain

// O(n^3), O(1)
const tigaN3 = (a) => {
  for (let i of a)
    for (let j of a)
      for (let k of a)
        if (i !== j && i !== k && j !== k && i + j === k * k) return true;
  return false;
};

// O(n log n), O(1)
const tigaCepat = (a) => {
  a = [...a].sort((x, y) => x - y);
  for (let k = 0; k < a.length; k++) {
    let l = 0,
      r = a.length - 1,
      t = a[k] * a[k];
    while (l < r) {
      if (l === k) l++;
      else if (r === k) r--;
      else {
        const s = a[l] + a[r];
        if (s === t) return true;
        s < t ? l++ : r--;
      }
    }
  }
  return false;
};

// TEST
console.log("A:", ukur(interN2, [1, 2, 3], [2, 3, 4]));
console.log("A cepat:", ukur(interSet, [1, 2, 3], [2, 3, 4]));

console.log("B:", groupAnagram(["eat", "tea", "tan", "ate", "nat", "bat"]));

console.log("C:", ukur(tigaN3, [1, 2, 3, 4, 5]));
console.log("C cepat:", ukur(tigaCepat, [1, 2, 3, 4, 5]));
```

Output:

```
A: { res: [ 2, 3 ], time: 0 }
A cepat: { res: [ 2, 3 ], time: 0 }
B: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
C: { res: true, time: 0 }
C cepat: { res: true, time: 1 }
```

13. Pada tahap berikutnya, buat sebuah file JavaScript bernama **tugas-2.js** di dalam folder **tugas** yang digunakan untuk mengerjakan tugas sesuai instruksi yang diberikan.

14. Selanjutnya lakukan penulisan kode program pada file **tugas-2.js** dengan mengikuti aturan dan instruksi dari tugas yang diberikan.

Code:

```javascript
// Fungsi-fungsi
const fn_O1 = (n) => n + 1; // O(1)

const fn_On = (n) => {
  // O(n)
  let sum = 0;
  for (let i = 0; i < n; i++) sum += i;
  return sum;
};

const fn_Onlogn = (n) => {
  // O(n log n)
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 1; j < n; j *= 2) count++;
  }
  return count;
};

const fn_On2 = (n) => {
  // O(n^2)
  let count = 0;
  for (let i = 0; i < n; i++) for (let j = 0; j < n; j++) count++;
  return count;
};

// ========================================
// Benchmark
function benchmarkSemua(ukuranData) {
  for (let n of ukuranData) {
    console.log(`\n=== n = ${n} ===`);

    let t0, t1;

    t0 = Date.now();
    fn_O1(n);
    t1 = Date.now();
    console.log("O(1):", t1 - t0, "ms");

    t0 = Date.now();
    fn_On(n);
    t1 = Date.now();
    console.log("O(n):", t1 - t0, "ms");

    t0 = Date.now();
    fn_Onlogn(n);
    t1 = Date.now();
    console.log("O(n log n):", t1 - t0, "ms");

    t0 = Date.now();
    fn_On2(n);
    t1 = Date.now();
    console.log("O(n^2):", t1 - t0, "ms");
  }
}

// ========================================
// RUN
benchmarkSemua([100, 500, 1000, 5000, 10000]);

// ========================================
// OBSERVASI (penting buat nilai)
/*
- O(1): waktu hampir tetap
- O(n): naik linear
- O(n log n): lebih cepat dari n^2 tapi lebih lambat dari n
- O(n^2): naik sangat cepat (paling lambat)

Kesimpulan:
Hasil sesuai teori Big-O (growth rate).
Semakin besar kompleksitas, semakin drastis kenaikan waktu.
*/
```

Output:

```
=== n = 100 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 2 ms

=== n = 500 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 5 ms

=== n = 1000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 4 ms

=== n = 5000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 78 ms

=== n = 10000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n^2): 308 ms
```

17. Pada tahap ini, buka Terminal / Git Bash**.** Langkah pertama adalah membuka aplikasi terminal seperti Git Bash untuk menjalankan perintah Git.

18. Selanjutnya, tambahkan file ke Staging area, tambahkan semua file ke staging area dengan perintah:
git add .

19. Pada tahap ini, Melakukan Commit. Menyimpan perubahan ke repository lokal dengan perintah:
git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"

```
User@LAPTOP-M9SKU557 MINGW64 ~/2025573010031_Algoritma_dan_Struktur_Data/praktikum-5 (main)
$ git status
On branch main
Your branch and 'origin/main' have diverged,
and have 2 and 9 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

nothing to commit, working tree clean

User@LAPTOP-M9SKU557 MINGW64 ~/2025573010031_Algoritma_dan_Struktur_Data/praktikum-5 (main)
$ git pull --rebase origin main
From https://github.com/zulpansahputra/2025573010031_Algoritma_dan_Struktur_Data
 * branch            main       -> FETCH_HEAD
Successfully rebased and updated refs/heads/main.

User@LAPTOP-M9SKU557 MINGW64 ~/2025573010031_Algoritma_dan_Struktur_Data/praktikum-5 (main)
$ git push origin main
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 8 threads
Compressing objects: 100% (14/14), done.
error: RPC failed; HTTP 408 curl 22 The requested URL returned error: 408
send-pack: unexpected disconnect while reading sideband packet
Writing objects: 100% (14/14), 2.83 MiB | 134.00 KiB/s, done.
Total 14 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
fatal: the remote end hung up unexpectedly
Everything up-to-date
```

BAB IV HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum

Dalam praktikum ini dilakukan beberapa pengujian algoritma, antara lain:

1. **Algoritma O(1)**
   Menggunakan rumus matematika untuk menjumlahkan angka.
   Hasil: sangat cepat (hampir 0 ms).
2. **Algoritma O(n)**
   Menggunakan perulangan untuk menjumlahkan angka.
   Hasil: lebih lambat dibanding O(1).
3. **Algoritma O(n^2)**
   Menggunakan nested loop untuk mencari pasangan angka.
   Hasil: sangat lambat untuk data besar.
4. **Binary Search (O(log n))**
   Mencari data dalam array terurut.
   Hasil: sangat efisien meskipun data besar.
5. **Perbandingan Space Complexity**
   - O(1): menggunakan variabel tetap
   - O(n): menggunakan array tambahan
6. **Refactoring Algoritma**
   - Nested loop (O(n^2)) diubah menjadi penggunaan Set (O(n))
   - Hasil: waktu eksekusi jauh lebih cepat

4.2 Pembahasan

Dari hasil praktikum dapat disimpulkan bahwa:

- Kompleksitas algoritma sangat berpengaruh terhadap performa program.
- Algoritma dengan kompleksitas **O(1)** adalah yang paling efisien.
- Penggunaan nested loop menyebabkan kompleksitas meningkat drastis menjadi **O(n^2)**.
- Penggunaan struktur data seperti **Set** dapat mengurangi kompleksitas dari O(n^2) menjadi O(n).
- Binary Search lebih efisien dibanding Linear Search karena menggunakan metode pembagian data.

Praktikum ini juga menunjukkan bahwa dua algoritma bisa menghasilkan output yang sama, tetapi memiliki performa yang sangat berbeda.

BAB V PENUTUP

5.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan mengenai analisis kompleksitas algoritma, dapat disimpulkan bahwa pemahaman terhadap Big O Notation sangat penting dalam dunia pemrograman. Big O membantu dalam mengetahui tingkat efisiensi suatu algoritma baik dari segi waktu (time complexity) maupun penggunaan memori (space complexity).

Dari hasil percobaan, terlihat bahwa algoritma dengan kompleksitas kecil seperti O(1) dan O(log n) memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas besar seperti $O(n^2)$ atau $O(2^n)$, terutama ketika jumlah data semakin besar. Selain itu, penggunaan struktur data yang tepat seperti Set atau Map dapat meningkatkan efisiensi algoritma secara signifikan.

Dengan demikian, pemilihan algoritma yang efisien sangat berpengaruh terhadap kinerja program secara keseluruhan.